

Homework 4 – Entropy Coding

Student: Chi-Yi, Lin ID: 314581003

1 Objective

This homework implements **entropy coding** for image compression using **Run-Length Encoding (RLE)**. A JPEG-like pipeline is applied on the grayscale image `lena.png`, including block-based DCT, quantization, zigzag scanning, RLE encoding/decoding, and inverse DCT reconstruction. The goal is to observe the trade-off between compression rate and image quality when using different quantization tables.

2 Methodology

2.1 8×8 DCT and IDCT

The image is divided into 8×8 blocks and transformed using the 2D DCT:

$$C(u, v) = \frac{1}{4} \alpha(u) \alpha(v) \sum_{x=0}^7 \sum_{y=0}^7 f(x, y) \cos \frac{(2x+1)u\pi}{16} \cos \frac{(2y+1)v\pi}{16} \quad (1)$$

Inverse DCT reconstructs each spatial block.

2.2 Quantization

The DCT coefficients are quantized to suppress high-frequency components:

$$Q(u, v) = \text{round} \left(\frac{C(u, v)}{Q_{\text{table}}(u, v)} \right)$$

Two quantization tables are used: Table 1 (fine quantization, better quality) and Table 2 (coarse quantization, stronger compression).

2.3 Zigzag Scanning

Each 8×8 block is flattened into a 1D array following the zigzag order, allowing low-frequency coefficients to appear first and trailing zeros to cluster at the end.

2.4 RLE Encoding and Decoding

Each sequence is RLE encoded as pairs (*run*, *value*), where *run* counts preceding zeros. The end of block is marked as EOB. Decoding reverses the process and restores the full coefficient matrix.

2.5 Image Reconstruction

Inverse zigzag → dequantization → IDCT → block recombination to form the final reconstructed image.

3 Implementation Workflow and Code Explanation

3.1 Overall Workflow

The implementation follows these sequential steps:

1. Load and convert image to grayscale.
2. Divide into 8×8 blocks.
3. Apply DCT and quantization.
4. Perform zigzag scanning.
5. Encode each block using RLE.
6. Decode, dequantize, and perform IDCT.
7. Merge all blocks to reconstruct the full image.

3.2 Main Script Structure

```
import numpy as np
from scipy.fftpack import dct, idct
from PIL import Image
import zlib, json

def block_process(img, func, block_size=8):
    h, w = img.shape
    out = np.zeros_like(img, dtype=np.float32)
    for i in range(0, h, block_size):
        for j in range(0, w, block_size):
            block = img[i:i+block_size, j:j+block_size]
            out[i:i+block_size, j:j+block_size] = func(block)
    return out

def dct2(block):
    return dct(dct(block.T, norm='ortho').T, norm='ortho')

def idct2(block):
    return idct(idct(block.T, norm='ortho').T, norm='ortho')
```

Listing 1: Main workflow in `vc_hw4_rle.py`

3.3 Quantization and Zigzag Scanning

```
def quantize(block, qtable):
    return np.round(block / qtable).astype(int)

def dequantize(block, qtable):
    return block * qtable

zigzag_index = np.array([
    [0,1,5,6,14,15,27,28],
    [2,4,7,13,16,26,29,42],
```

```

[3,8,12,17,25,30,41,43],
[9,11,18,24,31,40,44,53],
[10,19,23,32,39,45,52,54],
[20,22,33,38,46,51,55,60],
[21,34,37,47,50,56,59,61],
[35,36,48,49,57,58,62,63]
]).flatten()

def zigzag(block):
    return block.flatten()[zigzag_index]

def inverse_zigzag(array):
    block = np.zeros((8,8))
    block.flatten()[zigzag_index] = array
    return block

```

Listing 2: Quantization and Zigzag conversion

3.4 Run-Length Encoding and Decoding

```

def rle_encode(block):
    result = []
    run = 0
    for val in block[1:]:
        if val == 0:
            run += 1
        else:
            result.append((run, int(val)))
            run = 0
    result.append(('EOB',))
    return result

def rle_decode(sequence):
    result = [0]
    for item in sequence:
        if item[0] == 'EOB': break
        run, val = item
        result.extend([0]*run + [val])
    return np.array(result + [0]*(64 - len(result)))

```

Listing 3: RLE Encoding and Decoding

3.5 Compression and Reconstruction

```

def compress(img, qtable):
    h, w = img.shape
    encoded = []
    for i in range(0, h, 8):
        for j in range(0, w, 8):
            block = img[i:i+8, j:j+8] - 128
            q = quantize(dct2(block), qtable)
            encoded.append(rle_encode(zigzag(q)))
    return encoded

def decompress(encoded, qtable, shape):

```

```

h, w = shape
out = np.zeros(shape, dtype=np.float32)
idx = 0
for i in range(0, h, 8):
    for j in range(0, w, 8):
        rle_block = encoded[idx]; idx += 1
        q = inverse_zigzag(rle_decode(rle_block))
        out[i:i+8, j:j+8] = idct2(dequantize(q, qtable)) + 128
return np.clip(out, 0, 255)

```

Listing 4: Compression and Reconstruction Process

3.6 Compression Statistics

```

encoded_q1 = compress(img, QTABLE1)
json_bytes = json.dumps(encoded_q1).encode('utf-8')
compressed = zlib.compress(json_bytes)
print("Original bytes:", len(json_bytes),
      "Compressed bytes:", len(compressed))

```

Listing 5: File size and compression ratio

4 Experimental Setup and Results

Input image: 512×512 grayscale lena.png. Both quantization tables were tested.

Table 1: Compression results

Metric	QTable 1	QTable 2
RLE symbols	54068	38235
Raw bytes	356662	253127
Compressed (zlib)	46688	28262
Subjective quality	High	Lower (block artifacts)



Figure 1: Left: Original image; Middle: Reconstruction (QTable 1); Right: Reconstruction (QTable 2)

5 Discussion

Quantization Table 2 produces stronger compression because most high-frequency coefficients are quantized to zero. The resulting RLE sequence becomes shorter, but the reconstructed image exhibits visible loss of texture and smooth edges. Quantization Table 1 maintains better fidelity but with a larger file size. This behavior follows the **rate–distortion trade-off** principle.

6 Conclusion

This project successfully implemented block-based DCT compression with RLE entropy coding in Python. The system demonstrates that quantization strength directly affects both compression ratio and perceived quality. Future extensions may include DC differential encoding, Huffman coding, PSNR/SSIM analysis, and YCbCr color image support.

References

1. “JPEG Image Compression using DCT” – YouTube Tutorial, <https://www.youtube.com/watch?v=Q2aEzeMDHMA>
2. Q-Viper Blog, “Run-length encoding in Python”, <https://q-viper.github.io/2021/05/24/coding-run-length-encoding-in-python/>